

PART A
(PART A : TO BE REFERRED BY STUDENTS)
Experiment No. 04(A)

Aim: Write a program to implement Lexical Analyzer for given language using Finite Automata.

Objective: Develop a program to implement lexical analyzer using Finite Automata

Outcome: Students are able to design and implement lexical analyzer for given language.

Theory:

The very first phase of compiler is lexical analysis. The lexical analyzer read the input characters and generates a sequence of tokens that are used by parser for syntax analysis. The figure below summarizes the interaction between lexical analyzer and parser.

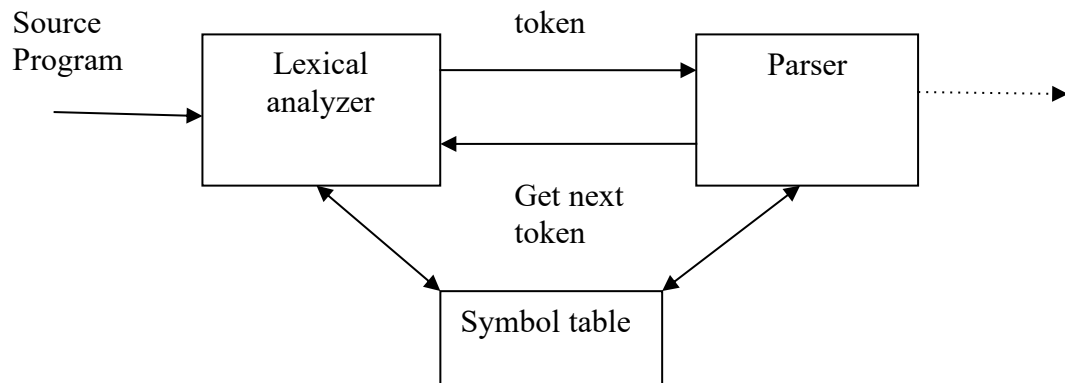


FIGURE: Interaction between lexical analyzer and parser

The lexical analyzer is usually implemented as subroutine or co-routine of the parser. When the “get next token” command received from parser, the lexical analyzers read input characters until it identifies next token.

Lexical analyzer also performs some secondary tasks at the user interface, such as stripping out comments and white spaces in the form of blank, tab and newline characters. It also correlates error messages from compiler to source program. For example lexical analyzer may keep track of number of newline characters and correlate the line number with an error message.

In some compilers, a lexical analyzer may create a copy of source program with error messages marked in it. An important notation used to specify patterns is a regular expression. Each pattern matches a set of strings. Regular expression will serve as a name for set of strings.

A *recognizer* for a language is a program that takes as input a string x and answer “yes” if x is a sentence of the language and “no” otherwise.

We compile a regular expression into a recognizer by constructing a generalized transition diagram called a *finite automaton*.

We will design lexical analysis for the language which consists of strings containing “abb” as substring.

Thus, $L = \{\text{“abb”}, \text{“babb”}, \text{“abbabba”}, \dots\}$

The DFA for this language is represented in transition table below:

Σ Q	a	b
q0	q1	q0
q1	q1	q2
q2	q1	q3
q3	q3	q3

Now we will simulate this DFA to generate lexical analyzer for the language L.

Finite automata is a state machine that takes a string of symbols as input and changes its state accordingly. Finite automata is a recognizer for regular expressions. When a regular expression string is fed into finite automata, it changes its state for each literal. If the input string is successfully processed and the automata reaches its final state, it is accepted, i.e., the string just fed was said to be a valid token of the language in hand.

The mathematical model of finite automata consists of:

- Finite set of states (Q)
- Finite set of input symbols (Σ)
- One Start state (q0)
- Set of final states (qf)
- Transition function (δ)

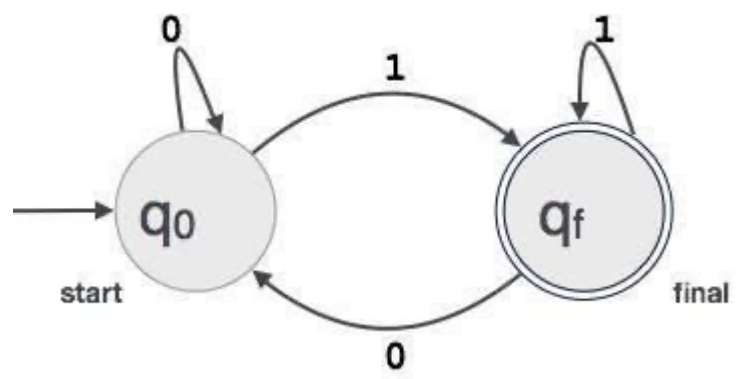
The transition function (δ) maps the finite set of state (Q) to a finite set of input symbols (Σ), $Q \times \Sigma \rightarrow Q$

Finite Automata Construction

Let L(r) be a regular language recognized by some finite automata (FA).

- **States** : States of FA are represented by circles. State names are written inside circles.
- **Start state** : The state from where the automata starts, is known as the start state. Start state has an arrow pointed towards it.
- **Intermediate states** : All intermediate states have at least two arrows; one pointing to and another pointing out from them.
- **Final state** : If the input string is successfully parsed, the automata is expected to be in this state. Final state is represented by double circles. It may have any odd number of arrows pointing to it and even number of arrows pointing out from it. The number of odd arrows are one greater than even, i.e. **odd = even+1**.
- **Transition** : The transition from one state to another state happens when a desired symbol in the input is found. Upon transition, automata can either move to the next state or stay in the same state. Movement from one state to another is shown as a directed arrow, where the arrows points to the destination state. If automata stays on the same state, an arrow pointing from a state to itself is drawn.

Example : We assume FA accepts any three digit binary value ending in digit 1. $FA = \{Q(q_0, q_f), \Sigma(0,1), q_0, q_f, \delta\}$



PART B

(PART B : TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per following segments within two hours of the practical. The soft copy must be uploaded at the end of the practical)

Roll. No. C-54	Name: Quazi Md Moinuddin
Class: TE-C	Batch: C-3
Date of Experiment: 22.03.2025	Date of Submission: 07.04.2025
Grade:	

B.1 Software Code written by student:

```
#include <stdio.h>
#include <conio.h>
#include <ctype.h>
#include <string.h>

void analyze(char str[]);

void main() {
    char input[100];
    int choice;

    clrscr(); // clear screen

    do {
        printf("Enter a line of input to analyze:\n");
        gets(input); // Read entire line
        analyze(input);

        printf("\nDo you want to analyze another line? (1 = Yes / 0 = No): ");
        scanf("%d", &choice);
        fflush(stdin); // clear input buffer

        clrscr(); // clear screen for next input

    } while (choice == 1);

    printf("\nThank you for using the Lexical Analyzer!\n");
    getch(); // wait for key press
}

// Function to identify and print tokens
void analyze(char str[]) {
    int i = 0;
    char ch;

    printf("\nLexical Tokens:\n");
```

```

while (str[i] != '\0') {
    ch = str[i];

    if (isspace(ch)) {
        i++;
        continue;
    }

    // Identifier or keyword
    if (isalpha(ch)) {
        char id[20];
        int j = 0;

        while (isalnum(str[i])) {
            id[j++] = str[i++];
        }
        id[j] = '\0';

        // You can expand this list for more keywords
        if (strcmp(id, "int") == 0 || strcmp(id, "float") == 0 || strcmp(id, "if") == 0 || strcmp(id,
"else") == 0)
            printf("<Keyword, %s>\n", id);
        else
            printf("<Identifier, %s>\n", id);
    }

    // Number
    else if (isdigit(ch)) {
        char num[20];
        int j = 0;

        while (isdigit(str[i])) {
            num[j++] = str[i++];
        }
        num[j] = '\0';
        printf("<Number, %s>\n", num);
    }

    // Operator
    else if (ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '=') {
        printf("<Operator, %c>\n", ch);
        i++;
    }

    // Separator
    else if (ch == ',' || ch == ';' || ch == '(' || ch == ')' || ch == '{' || ch == '}') {
        printf("<Separator, %c>\n", ch);
        i++;
    }
}

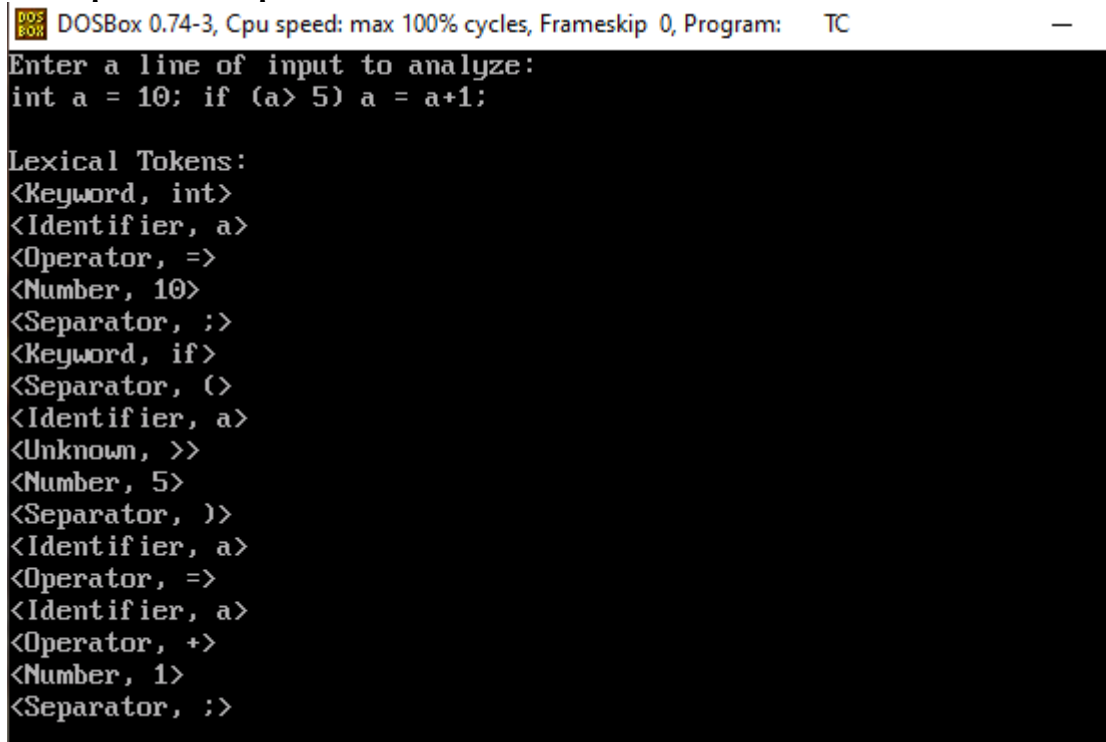
```

```

// Unknown character
else {
    printf("<Unknown, %c>\n", ch);
    i++;
}
}
}

```

B.2 Input and Output:



```

DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Program: TC
Enter a line of input to analyze:
int a = 10; if (a > 5) a = a+1;

Lexical Tokens:
<Keyword, int>
<Identifier, a>
<Operator, =>
<Number, 10>
<Separator, ;>
<Keyword, if>
<Separator, (>
<Identifier, a>
<Unknown, >>
<Number, 5>
<Separator, )>
<Identifier, a>
<Operator, =>
<Identifier, a>
<Operator, +>
<Number, 1>
<Separator, ;>

```

B.3 Observations and learning:

- Implemented a lexical analyzer using finite automata in Turbo C.
- Identified tokens like keywords, identifiers, numbers, and operators.
- Learned character handling and string comparison for token classification.
- Understood how lexical analysis works as the first phase of a compiler.

B.4 Conclusion:

The implementation of the lexical analyzer using finite automata successfully identified various tokens from the input. It provided a clear understanding of how lexical analysis simplifies the compilation process by breaking down source code into meaningful tokens. This foundational phase is crucial for the proper functioning of the subsequent stages of a compiler.

B.5 Question of Curiosity

1. What is token?

A **token** is the smallest unit in a programming language that has meaning, such as a keyword, identifier, operator, or symbol.

2. What is the role of lexical analyzer?

The **lexical analyzer** scans the source code and converts it into tokens by removing whitespaces, comments, and grouping characters into meaningful sequences.

3. What is the output of Lexical analyzer?

The **output** of a lexical analyzer is a **stream of tokens**, which is then passed to the syntax analyzer (parser) for further processing.